

```
<!--JAVASCRIPT-->
```

INVESTIGACION {

```
<Por="NAVARRO RAMIREZ GERSON  
ASAHEL" />
```

}



Características Básicas del Lenguaje {

JavaScript es el lenguaje de programación fundamental de la web moderna. Sus características principales definen su comportamiento y versatilidad:

- **Interpretado:** A diferencia de los lenguajes compilados (como C++ o Java), JavaScript no necesita un proceso de compilación previo para transformarse en código máquina. El navegador lee y ejecuta el código línea por línea directamente en tiempo de ejecución.
- **Dinámico (Tipado débil):** Las variables en JavaScript no están amarradas a un tipo de dato específico desde su creación. Una variable puede almacenar un número y, más adelante en el mismo programa, cambiar para almacenar un texto sin que esto genere un error.
- **Multiparadigma:** No obliga al desarrollador a programar de una sola forma. Permite adoptar diferentes estilos de programación según las necesidades del proyecto: Programación Orientada a Objetos (POO), Programación Imperativa o Programación Funcional.

}

Estructuras de Control{

Condicionales (if, else, switch):

Evalúan si una condición es verdadera o falsa para tomar caminos distintos.

Son las herramientas que permiten controlar el flujo de ejecución de un programa, decidiendo si ciertas líneas de código se ejecutan o si deben repetirse

Bucles (for, while, do-while):

Permiten repetir un bloque de código múltiples veces mientras se cumpla una condición específica, ideal para recorrer listas o automatizar tareas repetitivas

}

Tipos de Datos{

01

String:
Cadenas de
texto
("Hola").

02

Number:
Valores
numéricos,
tanto
enteros como
decimales.

03

Boolean:
Valores
lógicos
(true o
false).

04

Undefined:
Variable que
ha sido
declarada
pero aún no
tiene un
valor
asignado.

05

Null:
Representa
la ausencia
intencional
de cualquier
valor.

06

Objetos: No almacenan el valor
directamente, sino una referencia o
dirección de memoria hacia donde se
guardan los datos. Los tipos de
referencia incluyen a los Objetos
(Object), Arreglos (Array) y Funciones
(Function).

}

Funciones {

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. En JavaScript existen tres formas principales de definirlas:

Declaradas

Tienen un nombre explícito y sufren de hoisting (pueden ser llamadas en el código antes de la línea donde fueron creadas).
`function saludar() { return "Hola Mundo"; }`

Expresadas

Se definen dentro de una expresión (normalmente asignándolas a una variable). No tienen hoisting.
`const despedir = function() { return "Adiós"; };`

Flechas

Introducidas en las versiones modernas, ofrecen una sintaxis más corta y compacta. No tienen su propio contexto para la palabra clave `this`.
`const sumar = (a, b) => a + b;`

Objetos y Arrays {

Son las estructuras de datos complejas más utilizadas para organizar la información:

Objetos: Estructuras que representan entidades de la vida real mediante pares de "clave-valor" (propiedades y métodos).
`const coche = { marca: "Toyota", modelo: "Corolla", arrancar: function() {} };`

Arrays (Arreglos): Listas ordenadas de elementos (pueden ser números, textos u otros objetos) donde cada elemento tiene una posición numérica fija llamada índice, empezando desde el 0.

}

Eventos y Manipulación del DOM{

Esta es la parte que hace que las páginas web dejen de ser estáticas y se vuelvan interactivas:

- **DOM (Document Object Model):** Es la estructura de árbol que el navegador genera para representar el documento HTML. JavaScript utiliza el DOM para buscar, cambiar, agregar o eliminar elementos de la página web en tiempo real.
-
- **Eventos:** Acciones o sucesos que ocurren en el navegador. JavaScript "escucha" estos eventos y reacciona ejecutando código específico a través de funciones llamadas Event Listeners.

}

Asincronía{

JavaScript es de un solo hilo (ejecuta una cosa a la vez). Para evitar que la página se congele al hacer tareas tardadas (como pedir datos a un servidor), utiliza la asincronía:



CALLBACKS

Funciones que se pasan como argumentos a otra función para que se ejecuten una vez que la tarea pesada haya terminado. Es el método clásico.



PROMESAS

Objetos que representan el resultado futuro de una operación asíncrona. Pueden estar pendientes (Pending), resueltas con éxito (Fulfilled) o haber fallado (Rejected).



ASYNC

Una forma moderna y elegante de trabajar con promesas. Permite escribir código asíncrono que se lee y se comporta de manera muy similar al código síncrono tradicional, haciéndolo mucho más limpio.

JS

Características

Interpreted: Se ejecuta directamente en el navegador sin compilación previa.
Dinámico: Tipado débil, las variables cambian de tipo automáticamente.
Multiparadigma: Soporta programación funcional, orientada a objetos y funcional.

Estructuras de Control

Condicionales: if, else if, else, switch. (Ejemplo: Evaluar condiciones)
Bucles: for, while, do-while. (Ejemplo: Repetir tareas o recorrer colecciones)

Tipos de Datos

Primitivos: Almacenan un único valor directamente. (Ejemplos: String ("hola"), Number (42), Boolean (true), Undefined, Null, Symbol, BigInt)
Referencias: Almacenan direcciones de memoria estructuras complejas. (Ejemplos: Object, Array, Function)

Funciones

Declaradas: Usan la palabra function y tienen hoisting. (Ejemplo: function sumar(a, b) { return a + b; })
Expresadas: Se asignan a una variable. (Ejemplo: const restar = function(a, b) { return a - b; };)
Flechas (Arrow): Sintaxis compacta, no tienen su propio this. (Ejemplo: const mult = (a, b) => a * b;)

Objetos y Arrays

Objetos: Colecciones de propiedades y métodos (clave-valor). (Ejemplo: const usuario = { nombre: "Eva", edad: 29 };)
Arrays: Listas ordenadas de elementos indexados numéricamente. (Ejemplo: const colores = ["rojo", "azul", "verde"];)

Eventos y Manipulación del DOM

DOM (Document Object Model): Estructura en árbol que representa el HTML. (Ejemplo: document.getElementById("boton"))
Eventos: Acciones del usuario que el navegador detecta. (Ejemplo: elemento.addEventListener("click", miFuncion))

Asincronía

Callbacks: Funciones que se pasan como argumentos a otra función.
Promesas: Objetos que representan el éxito o fracaso de una operación futura.
Estados: Pending, Fulfilled, Rejected.
Async / Await: Azúcar sintáctico para manejar promesas de forma lineal/síncrona.